

Technical Stack and Architectural Foundation

The StockWise system is engineered upon a robust, scalable, and modular monolithic architecture, specifically designed to ensure high performance, maintainability, and data integrity in inventory management operations.

1.1 Programming Language & Framework: Java 21 & Spring Boot

- **Java 21 (LTS):** We have adopted Java 21 as the foundational programming language. As a Long-Term Support (LTS) release, it provides a stable environment, enhanced performance improvements, and modern language features that ensure the long-term viability of the StockWise system.
- **Spring Boot Framework:** The core business logic is developed using the Spring Boot framework. This choice enables a "convention-over-configuration" approach, significantly reducing boilerplate code and allowing the development team to focus on implementing inventory-specific features.
- **Production-Ready Ecosystem:** Through Spring Boot's extensive ecosystem, we leverage **Spring Web** to handle HTTP requests and **Spring Security** to handle comprehensive authentication and authorization workflows, ensuring that our application is both performant and secure.

1.2 Database Management: PostgreSQL

- **Relational Integrity:** PostgreSQL is implemented as the primary Relational Database Management System (RDBMS). It was selected for its strict ACID (Atomicity, Consistency, Isolation, Durability) compliance, which is critical for inventory systems where data accuracy (e.g., current stock counts, transaction history) cannot be compromised.
- **Query Performance:** PostgreSQL excels at handling complex relational queries, which is essential for our requirement of generating analytical reports and managing relationships between `Product` entities, `Categories`, and `Inventory Transactions`. Its ability to scale horizontally and vertically ensures that the system remains responsive as the product catalog grows.

1.3 ORM & Data Access: Spring Data JPA (Hibernate)

- **Object-Relational Mapping (ORM):** To bridge the gap between our object-oriented Java models and the relational database, we utilize **Spring Data JPA** (implementing the Hibernate framework).
- **Repository Pattern:** This abstraction layer allows us to interact with the database using the Repository pattern, which simplifies data access by providing built-in methods for CRUD (Create, Read, Update, Delete) operations.
- **Security & Efficiency:** Spring Data JPA minimizes the risk of SQL injection by utilizing prepared statements automatically. It also optimizes database interactions through efficient query generation, ensuring that the backend remains lean and performant even under heavy concurrent loads.

1.4 Application Architecture: Thymeleaf & Server-Side Rendering (SSR)

- **Monolithic Architecture:** The system employs a monolithic architecture using the **Model-View-Controller (MVC)** design pattern. This ensures that the application logic, data handling, and UI rendering are managed within a unified, cohesive codebase.
- **Thymeleaf Template Engine:** We utilize **Thymeleaf** as our server-side rendering engine. This allows us to dynamically inject data into HTML templates on the server before serving the final page to the browser. This approach simplifies state management, as the server maintains the application context, and ensures a seamless, highly secure user experience without the complexity of client-side JavaScript frameworks.
- **Integrated Security:** By rendering the UI on the server, we enforce security controls (such as role-based page access) directly at the controller level, ensuring that sensitive data is only processed and displayed based on the user's authorized privileges.